

Pipelined Processor Design

Instruction Set

Opcode	Instruction
00000	RESERVED SIGNAL
00001	NOP
00010	HLT
00011	SETC
00100	NOT
00101	INC
00110	OUT
00111	IN
01000	_____
01001	MOV
01010	SWAP
01011	ADD
01100	SUB
01101	AND
01110	IADD
01111	_____
10000	_____
10001	PUSH
10010	POP
10011	LDM
10100	LDD

Opcode	Instruction
10101	STD
10110	_____
10111	_____
11000	JZ
11001	JN
11010	JC
11011	JMP
11100	CALL
11101	RET
11110	INT
11111	RTI

Register file → 3-bit selector

Register	Encoding
R0	0000
R1	0001
R2	0010
R3	0011
R4	0100
R5	0101
R6	0110
R7	0111

Instruction Format

R-type

[31:27] opcode
[26:24] Rdst
[23:21] Rsrc1
[20:18] Rsrc2
[17:0] 000...0

I-type

[31:27] opcode
[26:24] Rdst
[23:21] Rsrc
[20:0] 000...0

immediate
[31:0] imm/offset

One-operand

[31:27] opcode
[26:24] Rdst
[23:0] 000...0

Jump

[31:27] opcode
[26:0] 000..0

immediate
[31:0] imm

Hazards and their Solutions

Structural Hazards

Register File Access

- Reading from register file at decode stage, and writing into it at the write back stage
- **Solution:** write at rising edge and read at falling edge

Memory Access

- Reading from memory at fetch stage, and writing/reading from it at the memory stage
- **Solution:**
 - Stall the pipeline for one cycle; do not fetch a new instruction until memory access is complete
 - Hazard detection unit detects this hazard when **memory read** signal is 1

Data Hazards

Instruction Depends on a Register Evaluated in the Execute Stage

- Instruction depends on a register that a former instruction writes back the execute stage result
- **Solution:**
 - **Consecutive instructions:** forward data from execute stage
 - **One instruction in between:** forward data from memory stage
 - **More than one instruction gap:** no hazard

Instruction Depends on a Register Evaluated in the Memory Stage

- Instruction depends on a register that a former instruction writes back the memory stage result
- **Solution:**

- **Consecutive instructions:** stall one cycle, then forward from memory stage
- **One instruction in between:** forward from memory stage
- **More than one instruction gap:** no hazard

Notes

- The forwarding unit detects hazards by comparing **Rsrc1** and **Rsrc2** of the current instruction with **Rdst** of instructions in the execute and memory stages. If a match is found and the write-back signal is active, forwarding occurs.
- There is a hazard detection unit that detects load use-case and similar instructions (any instruction that reads from memory and write back into the register file). This unit compares **Rsrc1**, **Rsrc2** with **Rdst** of the instruction at the execute stage, and check if the memory read and write back signals are active.

Control Hazards

Branch and Change of Control Operations

- Branches are predicted as not taken, which may allow some instructions to enter the pipeline and later require flushing.
- Other control operations don't get the address to jump to until the execute stage
- **Solution:**
 - If any branch decision or change of control operation is taken, reset the **IF/ID** and **ID/EX** buffers
 - For RET and RTI instructions, the target PC is determined at the memory stage, so the **EX/MEM** buffer must also be reset in these cases.

Detailed Control Design

